ASE CHAPTER 7

T3 Comparison & Analytical Breakdown Pack

Comparison 7.1 · Hardware Reliability vs Software Reliability (Exercise 7.3)

Compare hardware reliability with software reliability. Why does software not experience physical wear-out?

While both hardware and software reliability deal with the probability of failure-free operation, they differ fundamentally in failure causes, degradation mechanisms, and maintenance impacts:

- **Nature of Failures:**
- **Hardware failures are predominantly physical:** Caused by physical wear-out, chemical degradation, manufacturing defects, thermal stress, and environmental fatigue.
- **Software failures are strictly design-related:** Software is a logical entity. Software failures occur solely due to **unmanifested design, logic, specification, and coding faults** activated by specific operational input sequences.
- **Failure Trajectory Over Time:**
- **Hardware follows the classic Bathtub Curve:** Characterized by high burn-in infant mortality, an extended constant-hazard operational phase, and a sharp upward wear-out period due to physical degradation.
- **Software does not wear out:** Software possesses no physical moving parts, does not rust, and suffers zero wear-out. **Ideally, software failure rate decreases monotonically over time as faults are identified and fixed.** In practice, software failure intensity exhibits sudden upward spikes whenever code changes or maintenance activities introduce regression faults, followed by subsequent burn-in.
- **Effect of Maintenance and Repairs:**
- **Hardware repair restores original reliability:** Replacing a worn-out component with a fresh spare part restores the machine to its previous operating baseline.
- **Software "repair" changes system design:** Software maintenance does not replace identical physical parts; **it alters the code logic itself.** A software bug fix can introduce new subtle faults, altering the program structure and potentially lowering overall system reliability.
- **Manufacturing and Duplication:**
- Hardware reliability varies between units due to manufacturing variances and tolerances.
- Software duplication is bit-for-bit exact; if one copy of a software release has a latent defect, every identical copy contains that exact same defect.

ملاحظة أكاديمية هامة (Academic Note)

الفارق الجوهري: العتاد يبلى فيزيائياً مع الزمن (Wear-out) واستبدال القطعة يعيده لحالته الأصلية ← البرمجيات لا تبلى، لكن "إصلاحها" هو تعديل تصميمي قد يدخل عيوباً جديدة (Regression faults) تغير هيكل النظام.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.1, pp. 308–310; companion slides 2–5.

Comparison 7.2 · Software Quality vs Software Reliability (Exercise 7.22)

Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss them.

Although software quality and software reliability are closely linked quality attributes, they differ in conceptual breadth, operational measurement, and time dependency:

- **Conceptual Scope (Umbrella vs Component):**
- **Software Quality is an overarching, multi-dimensional umbrella concept:** It encompasses the totality of attributes that satisfy customer requirements and business needs, including Functionality, Usability, Efficiency, Maintainability, Portability, and Reliability.
- **Software Reliability is a specific, focused attribute of quality:** It focuses strictly on failure-free operational execution over specified time intervals.
- **Time Dependency:**
- **Quality can be assessed statically without execution:** Aspects of quality such as code readability, modularity, architectural documentation, and structural complexity exist and can be measured independently of runtime.
- **Reliability is strictly dynamic and time-dependent:** Reliability cannot exist in the abstract without execution time or operational usage; it is fundamentally a probability defined over an operational time interval ($[0, t]$).
- **Measurement Metric:**
- **Quality is assessed through diverse qualitative and quantitative criteria:** Function points, cyclomatic complexity, usability surveys, and compliance audits.
- **Reliability is evaluated through precise stochastic and statistical indicators:** Failure intensity (λ), Mean Time Between Failures (MTBF), Mean Time To Failure (MTTF), and cumulative failure distributions ($\mu(t)$).
- **Operational Independence:**
- A software system can be mathematically reliable (never crashes because it executes a stable loop) while having poor quality (terrible user interface, high computational resource consumption, impossible to modify).
- Conversely, software may possess high internal quality (modular, well-documented, clean architecture) but suffer low operational reliability if key boundary-condition bugs crash the system under heavy operational load.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

خلاصة المقارنة: الجودة مفهوم كلي يشمل الموثوقية ← يمكن قياس أبعاد الجودة استاتيكياً (مثل سهولة القراءة والتعقيد)، بينما الموثوقية مرتبطة حصراً بالتشغيل الديناميكي والزمن والاحتمالية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2, pp. 321–323; companion slides 24–29.

Comparison 7.3 · ISO 9126 vs McCall Software Quality Model (Exercise 7.14)

Compare ISO 9126 with the McCall software quality model and highlight the key advantages of ISO 9126.

Both McCall (1977) and ISO 9126 (1991) establish hierarchical models to evaluate software product quality, but they exhibit structural and methodological differences:

- **Historical Context & Origin:**
- **McCall Model:** Developed for the US Air Force and military procurement; primarily oriented toward defense contracts and traditional waterfall lifecycles.
- **ISO 9126:** Developed by international consensus through the International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC) as a universal commercial standard.
- **Top-Level Structural Organization:**
- **McCall organizes 11 factors into 3 operational perspectives:** Product Operation (5 factors), Product Revision (3 factors), and Product Transition (3 factors).
- **ISO 9126 organizes quality into 6 distinct characteristics:** Functionality, Reliability, Usability, Efficiency, Maintainability, and Portability.
- **Decomposition Depth:**
- McCall presents 11 quality factors and 25 quality criteria.
- ISO 9126 decomposes 6 characteristics into 21 clearly delineated sub-characteristics (attributes), establishing a clean boundary between external user-visible behaviors and internal code properties.

Key Advantages of ISO 9126 over the McCall Model:

1. **Elimination of Factor Overlap:** In McCall's model, criteria like *Simplicity* and *Consistency* overlap across multiple factors. ISO 9126 establishes strict non-overlapping boundaries among characteristics to ensure orthogonal evaluation.
2. **Explicit Functionality Characteristic:** McCall distributes functional aspects across correctness and integrity without an explicit top-level characteristic. ISO 9126 explicitly elevates **Functionality** (Suitability, Accuracy, Interoperability, Compliance, Security) as a primary pillar of quality.
3. **International Standardization and Harmonization:** ISO 9126 represents broad global consensus across industries, whereas McCall represents early US defense procurement guidelines.
4. **Modern Lifecycle Adaptability:** ISO 9126 cleanly distinguishes between process quality, internal product metrics, external product metrics, and quality-in-use.

ملاحظة أكاديمية هامة (Academic Note)

مزايا ISO 9126 على مكول: 1. جعل "الوظائفية" (Functionality) خاصية عليا مستقلة.
2. إزالة التداخل المعياري بين العوامل وتحقيق التعامد (Orthogonality).
3. اعتماد عالمي موحد متوافق مع مختلف قطاعات الصناعة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1, Fig. 7.10, p. 322; Table 7.5(b), pp. 323–324; §7.2.4, pp. 326–328; companion slides 35–46.

Comparison 7.4 · Boehm Software Quality Model vs McCall Model

Compare Barry Boehm's software quality model with the McCall software quality model.

Both Boehm and McCall organize software-quality attributes hierarchically, but their model structures differ:

- **Top-Level Perspective:**
- **McCall's Top Level:** Focuses on **operational perspectives of software use and maintenance** (Operation, Revision, Transition).
- **Boehm's Top Level:** General Utility links directly to **Portability** and **Maintainability**. As-is Utility links to Reliability, Efficiency, and Human Engineering; Maintainability links to Testability, Understandability, and Modifiability.
- **Hierarchical Level Decomposition:**
- **McCall Model:** Uses a 3-tier hierarchy: **Quality Factors** → **Quality Criteria** → **Metrics**.
- **Boehm Model:** The seven intermediate constructs are **Portability, Reliability, Efficiency, Human Engineering, Testability, Understandability, and Modifiability**. The model lists 12 primitive characteristics, including device independence, completeness, accuracy, consistency, device efficiency, accessibility, communicativeness, self-descriptiveness, structuredness, conciseness, legibility, and augmentability.
- McCall's model organizes 11 quality factors under the three product perspectives and further relates them to criteria and metrics.

ملاحظة أكاديمية هامة (Academic Note)

الفارق المعماري: يربط الشكل General Utility مباشرةً بـ Portability وMaintainability؛ ويربط As-is Utility بـ Reliability وEfficiency وHuman Engineering، وMaintainability بـ Testability وUnderstandability وModifiability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1–§7.2.2, pp. 324–326; companion slides 40–41.

Comparison 7.5 · Basic Execution Time Model vs Logarithmic Poisson Execution Time Model (Exercise 7.17)

Explain the Basic and Logarithmic Poisson software reliability models and their significance in reliability studies.

The Basic Execution Time Model and the Logarithmic Poisson Execution Time Model are execution-time reliability growth models with different assumptions and failure-intensity functions:

- **Failure Intensity vs Failures Experienced:**
- **Basic Model: Linear relationship.** Failure intensity decreases linearly with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

$$\frac{d\lambda}{d\mu} = -\frac{\lambda_0}{v_0}$$

The model gives a constant linear decrease in failure intensity per failure experienced.

- **Logarithmic Poisson Model: Exponential relationship.** Failure intensity decreases exponentially with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

The absolute reduction per additional failure becomes smaller as failure intensity falls, because the reduction is proportional to the current intensity.

- **Total Failures Over Infinite Time (v_0):**
- **Basic Model: Finite.** Assumes a finite total expected number of failures v_0 over unbounded execution time.
- **Logarithmic Poisson Model: Infinite.** Its model form has an infinite expected number of failures over unbounded execution time ($v_0 = \infty$).
- **Failure Intensity vs Execution Time (τ):**
- **Basic Model:** Decays **exponentially** with execution time:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

- **Logarithmic Poisson Model:** Decays **inversely** with execution time:

$$\lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau + 1}$$

- **Significance in Reliability Studies:**
- The models represent different assumptions about failure-intensity reduction and the total expected number of failures. Which one is more suitable depends on how well its assumptions match the system and observed testing data; neither is universally superior.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة محورية: - الأساسي (Basic): تناقص خطي مع الأخطاء (μ)، وعدد الأخطاء الكلي محدود ($v_0 < \infty$). - اللوغاريتمي (Logarithmic): تناقص أسي مع الأخطاء (μ)، وعدد الأخطاء عبر الزمن اللانهائي غير محدود ($v_0 = \infty$).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1 & §7.3.2, pp. 329–339; companion slides 47–71.

Comparison 7.6 · Jelinski–Moranda Model vs Musa Basic Execution Time Model

Compare the Jelinski–Moranda reliability model with Musa's Basic Execution Time Model.

Both Jelinski–Moranda and Musa Basic are finite-failure reliability growth models, but they represent the failure process differently:

- **Time Representation — Jelinski–Moranda:** Represents the hazard for each interval between successive failure events; the model's hazard changes after a failure is removed.
- **Time Representation — Musa Basic Model:** Defines failure intensity against execution time (τ, CPU time) and failures experienced. A separate calendar-time component can relate execution time to elapsed calendar time.
- **Hazard Function Structure:**
- **Jelinski–Moranda:** Defines a **discrete, piecewise constant hazard rate** during the inter-arrival interval between the $(i-1)$-th and i-th failure:

$$\lambda(t_i) = \phi[N - (i-1)]$$

  The hazard rate graph is a discrete descending staircase.
- **Musa Basic Model:** Formulates failure intensity as a **continuous function of execution time** and cumulative failures experienced:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right), \quad \lambda(\mu) = \lambda_0\left(1 - \frac{\mu}{v_0}\right)$$

- **Fault-to-Failure Relationship:** Jelinski–Moranda assumes one fault is removed after each observed failure. Musa Basic represents progress through failures experienced (μ) and the finite expected-failure parameter (v_0).

📝 **ملاحظة أكاديمية هامة (Academic Note)**

وجه الشبه والاختلاف: كلا النموذجين يفترضان عدداً محدوداً من الأخطاء وتناقصاً ثابتاً لشدة الفشل مع إزالة العيوب ← نموذج Jelinski–Moranda يغيّر الشدة على خطوات بعد كل فشل، بينما Musa Basic يعبّر عنها بدلالة زمن التنفيذ وعدد حالات الفشل.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1 & §7.3.4, pp. 329–334, 343–346; companion slides 47–58, 89–92.

Comparison 7.7 · Mills Bug Seeding vs Software Reliability Growth Models (SRGMs)

Compare the Mills bug seeding technique with analytical Software Reliability Growth Models (such as Musa and Jelinski–Moranda).

Bug seeding estimates a remaining fault count from discovery counts, whereas software reliability growth models describe failure behavior over execution time:

- **Input Data Required:**
- **Bug Seeding:** Uses the number of seeded faults (S), detected seeded faults (s), and detected indigenous faults (n). It does not use elapsed execution time in the basic estimate.
- **SRGMs:** Use observed failure counts and execution-time or failure-time data, depending on the model.
- **Output Metrics Provided:**

- **Bug Seeding:** Estimates a total indigenous-fault count ($\hat{N}$), from which an estimated remaining count can be obtained. The estimate is not a failure-intensity forecast.
- **SRGMs:** Estimate or predict failure intensity and reliability-growth behavior as testing proceeds.
- **Practical Vulnerabilities:**
- Bug seeding assumes seeded and indigenous bugs have comparable chances of detection. If seeded bugs are easier to find, the estimate can be biased downward.
- Reliability-growth models use model-appropriate failure and time observations. Musa's calendar-time estimates additionally require resource and calendar-time data.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة منهجية: - زراعة الأخطاء (Bug Seeding): تستخدم أعداد الأخطاء المزروعة والمكتشفة لتقدير العدد الكلي للأخطاء الأصلية، ثم المتبقي منها. - نماذج نمو الموثوقية (SRGMs): تربط بيانات الفشل بزمن التنفيذ لتقدير شدة الفشل وسلوك نمو الموثوقية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1–§7.3.5, pp. 329–347; companion slides 47–93.

Comparison 7.8 · SEI CMM vs ISO 9001: Structural Philosophy & Differences (Exercise 7.31 & 7.34)

List the differences between CMM and ISO 9001. Why is it suggested that CMM is the better choice for software organizations?

The two frameworks overlap, but differ in purpose and emphasis:

Scope: ISO 9001 is a general quality-management standard for many industries. CMM is focused on software development; it can be applied to development and maintenance, although it does not directly address every maintenance-specific concern.
Emphasis: ISO 9001 sets minimum criteria for an adequate quality-management system. CMM places stronger emphasis on continuous software-process improvement and offers more detailed guidance for software organizations.
Shared principle: Both emphasize documenting the process and following the documented process.
Practical choice: CMM is the more detailed guide when the goal is software-process improvement. ISO 9001 certification may still be needed when a customer or market requires it. The goal should be process improvement, not the certificate or maturity score.

Overall, ISO 9001's general concerns are largely encompassed by CMM, while ISO 9001 does not cover all of CMM's software-process-improvement concerns.

ملاحظة أكاديمية هامة (Academic Note)

CMM أدق لتحسين عمليات البرمجيات، وISO 9001 يفيد عند الحاجة إلى شهادة يطلبها السوق أو الزبون. التداخل بينهما لا يعني أنهما متطابقان.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.4–§7.5.3, pp. 352–357; companion slides 97–107.

Comparison 7.9 · ISO 9001 Certification vs SEI CMM-Based Evaluation (Exercise 7.35)

Discuss the relative merits of ISO 9001 certification and the SEI CMM-based evaluation. Point out some of the shortcomings of the ISO 9001 certification process as applied to the software industry.

Relative merits and limits:

ISO 9001: It provides a recognized quality-management certification and a minimum set of criteria for an adequate quality system. A customer or market may require the certificate.
CMM: It focuses on software processes, stresses continuous improvement, and gives more detailed guidance for improving software organizations.
ISO 9001 in software: Its general scope means its requirements need interpretation for software work. It defines an adequate quality system, but does not provide the same detailed software-process-improvement guidance as CMM.
CMM evaluation: A maturity score is useful as a process-improvement reference, but achieving a level should not replace the goal of improving the process itself.

Conclusion: Use CMM when the main objective is detailed software-process improvement, and address ISO 9001 where certification is required. The frameworks overlap substantially, but neither the certificate nor a maturity score is the improvement goal.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ميزة CMM هي التوجيه التفصيلي للتحسين المستمر؛ وقد تبقى شهادة ISO مطلوبة تجارياً. لا تساوي الشهادة مستوى نضج CMM، ولا يُعدّ أيٌّ منهما غاية التحسين.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.4–§7.5.3, pp. 352–357; companion slides 103–107.

Comparison 7.10 · Relationships Between ISO 9001:1994 Clauses and CMM Practices (Exercises 7.29 & 7.30)

Discuss the correlation between the 20 clauses of ISO 9001 (1994) and the Key Process Areas (KPAs) of the SEI Capability Maturity Model.

The relationships are not one-to-one. One ISO clause may involve several CMM practices, and some ISO concerns are outside CMM's scope. The following are the clause-to-practice correspondences:

1. Management responsibility: Primarily Software Quality Assurance (SQA); Software Project Planning (SPP) and Software Project Tracking and Oversight (SPTO) also establish project responsibilities.
2. Quality system: Primarily SQA. Relevant procedures are distributed across Activities Performed practices; SPP specifies project procedures, and SQA and Verifying Implementation check compliance.
3. Contract review: Requirements Management clarifies software requirements allocated from customer needs.
4. Design control: Software Product Engineering (SPE) covers requirements analysis, design, coding, and testing; SPP plans these activities, SPTO tracks them, and Software Configuration Management (SCM) controls their work products.

5. Document control: SCM controls document distribution and modification; specific project procedures and standards are distributed across practices.
6. Purchasing: Software Subcontract Management addresses the supplier relationship.
7. Purchaser-supplied product: Integrated Software Management addresses purchased, off-the-shelf, or reusable software; coverage in this area is weak.
8. Product identification and traceability: Primarily SCM.
9. Process control: Procedures are distributed across CMM practices; project-specific procedures and standards are specified in the software development plan.
10. Inspection and testing: Product testing is in SPE; software-process inspections are addressed by Peer Reviews.
11. Inspection, measuring, and test equipment: Generally addressed through testing practices in SPE.
12. Inspection and test status: Testing practices in SPE address test status; problem reporting and configuration status are addressed through SCM.
13. Control of nonconforming product: Design, implementation, testing, and validation are addressed in SPE; SCM tracks configuration-item status and known unfixed defects. Installation is not addressed.
14. Corrective action: Field-defect analysis and corrective action are handled through problem reporting and controlled maintenance of baseline work products.
15. Handling, storage, packaging, and delivery: CMM does not cover replication, delivery, or installation.
16. Quality records: Record-maintenance practices are distributed across CMM practices.
17. Internal quality audits: SQA addresses the audit process; specific audits are included in the auditing practice of the Verifying Implementation common feature.
18. Training: Training needs and training records are addressed by Training and Orientation practices in the Ability to Perform common feature.
19. Servicing: Maintenance is embedded across CMM practices, but its unique concerns are not directly addressed as a separate process area.
20. Statistical techniques: Measurement practices are distributed across CMM; product measurement appears among Activities Performed practices, while process measurement is addressed by the Measurement and Analysis common feature.

Overall, the frameworks overlap substantially, but do not form an exact route map. ISO 9001's general quality-system concerns are largely encompassed by CMM; the reverse is less true. Some specific ISO concerns remain outside CMM, so the frameworks should not be treated as equivalent.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **العلاقة تقريبية وليست بنداً مقابل بند. بعض المتطلبات موزعة بين أكثر من ممارسة، وبعضها مثل التسليم لا يغطيه CMM؛ لذلك لا يصح تحويل التداخل إلى نسبة امتثال أو مساواة كاملة.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.5.1–§7.5.3, pp. 353–357; companion slides 103–107.
